



CENTRO UNIVERSITÁRIO JORGE AMADO

Allan Gabriel da Silva Barbosa Santos
Ednei dos Santos Nascimento Junior
Gabriel Marques Rosário dos Santos

RELATÓRIO – ATIVIDADE AVALIATIVA III

Salvador

2026

Allan Gabriel da Silva Barbosa Santos
Ednei dos Santos Nascimento Junior
Gabriel Marques Rosário dos Santos

RELATÓRIO – ATIVIDADE AVALIATIVA III

Atividade avaliativa apresentada ao Centro
Universitário Jorge Amado, como parte dos
requisitos necessários à obtenção do título de
Tecnólogo em Análise e Desenvolvimento de
Sistemas.

Professor: Taijara Santana
Disciplina: Estrutura de Dados

Salvador
2026

RESUMO

Este documento compõe a documentação técnica referente a terceira atividade avaliativa utilizando o escopo prático de estrutura de dados em linguagem C, a aplicação configura um jogo de estratégia em matriz executado de forma síncrona diretamente no terminal, simulando um ambiente dinâmico onde o usuário manipula um ladrão em fuga, confrontado por obstáculos estáticos e por um policial com lógica de perseguição automatizada que simula o comportamento de uma inteligência artificial (IA).

Palavras-chave: Matriz, Alocação Dinâmica de Memória, Manipulação de Arquivos em C, Estruturas de Dados, Ponteiros, Structs.

SUMÁRIO

1 INTRODUÇÃO	4
2 CONCEITO	5
3 ESTRUTURAS UTILIZADAS	6
3.1 Menu	6
3.2 Mapa	6
3.3 Jogador	7
3.4 Policial	8
3.5 Carro	8
4 MOVIMENTAÇÃO	10
4.1 Jogador	10
4.2 Policial	10
5 GAME OVER	12
6 JUSTIFICATIVAS DAS ESCOLHAS	13
7 PRINCIPAIS DIFICULDADES	14
REFERÊNCIAS	15

1 INTRODUÇÃO

A linguagem de programação C, apesar de não ser uma linguagem recente, possui relevância histórica e foi capaz de permitir que a tecnologia seja tão evoluída como é atualmente. Ela foi a base para a criação de diversos sistemas conhecidos mundialmente como o Windows, Linux e até mesmo o sistema operacional do Playstation 2 por agir em baixo nível computacional e ter o poder de manipular memória diretamente, também oferecendo o cenário ideal para programação de sistemas, desenvolvimento de software embarcado e aplicações nas quais a alta performance e o controle minucioso de recursos de hardware são essenciais. A utilização desses conceitos é essencial para aprender como um sistema funciona desde a sua raiz, e a pauta desse projeto apresenta isso na prática.

2 CONCEITO

Este projeto em questão foge da lógica de simulação inicialmente proposta e segue por um caminho completamente diferente enquanto mantém sua complexidade, se tornando um jogo interativo onde o jogador foge de uma entidade com uma IA (Inteligência Artificial) simples classificada como Policial, aumenta a própria pontuação enquanto evita obstáculos e foge em um carro após alcançar a quantidade mínima de pontos. Foram utilizados os conceitos de *struct*, alocação dinâmica de memória, ponteiros, leitura de arquivo externo e matriz dinâmica, além da adição de uma lógica de perseguição adicionada ao policial, na qual se chama *Distância de Manhattan*.

O jogador inicia em um local específico no mapa, no qual existem obstáculos, moedas e um policial que segue todos os passos do jogador, encurtando a distância entre ambos a cada turno de forma dinâmica. Após a pontuação ser aumentada para 10 com a coleta das moedas espalhadas pelo cenário, é adicionado um carro ao mapa, no qual o jogador precisa alcançar para conseguir finalizar o jogo, tudo enquanto continua evitando o policial e o *Game Over* que viria em seguida caso o mesmo o alcance.

3 ESTRUTURAS UTILIZADAS

O projeto teve como prioridade utilizar estruturas trabalhadas em sala, mantendo a sua simplicidade enquanto mantém o nível de complexidade solicitado. Foram utilizados conceitos específicos em cada seguimento do código, no qual serão destrinchados a seguir.

3.1 Menu

Para a confecção do menu, foi utilizado o conceito de *Switch/Case*, no qual o usuário digita 1 para iniciar o jogo ou 0 para limpar a memória alocada e encerrar o programa com segurança. A resposta do usuário é registrada na variável “menu”.

Após a função *scanf*, utilizado para captar a entrada do usuário, é aplicado o comando “*while(getchar() != '\n');*”, que é utilizado para limpar o buffer de entrada, apagando o *enter* que o usuário digitar e evitando comportamentos inesperados nas próximas linhas de código.

Sendo inserido o número um no console, é iniciado um loop *while()*, no qual são iniciadas as seguintes funções: *limparTela()*, *pontuacao()* e *renderizarMapa()*. Essas funções são executadas até o encerramento do programa e limpam o terminal a cada turno (evitando poluição visual), informam a pontuação do usuário em cada turno e lê o mapa novamente com as novas mudanças, respectivamente.

Caso o usuário digite um caractere que não seja 1 ou 0, é exibida uma mensagem de erro, a memória alocada é liberada e o programa é encerrado.

3.2 Mapa

O mapa do jogo é criado a partir de um arquivo nomeado como “nivel1.txt”, que é composto por dois números na primeira linha do arquivo, seguidos por pontos, “1” e “\$” nas linhas seguintes. Utilizando a função *fscanf*, os números da primeira linha são lidos e registrados em duas variáveis globais chamadas “altura” e “largura”. Após o registro dos valores, é alocada memória o suficiente no ponteiro mapa, que, por sua vez, é interpretado como uma matriz dinâmica. A partir do momento em que o mapa não é uma matriz estática, que seria declarada como “mapa[20][20]”, ele é uma matriz dinâmica, representada como mapa[linha * larguraTotaldaMatriz +

coluna]. Essa fórmula nos mostra como uma matriz é lida por extenso, já que a matriz dinâmica simula um ambiente 2D sendo escrito como 1D.

Os caracteres seguintes aos dois primeiros números do arquivo do mapa, sendo eles “.”, “\$” e “1” representam espaços vazios em que o jogador e o policial podem transitar, moedas a serem coletadas e paredes intransponíveis no qual ambos devem evitar, respectivamente. O arquivo é fechado logo após a primeira leitura, fazendo com que o resto do processo seja lido sem a dependência do arquivo de texto e evitando a possibilidade de que o arquivo seja corrompido caso seja fechado de forma indevida. Esses caracteres são lidos a partir da função *renderizarMapa()*, sendo ela um laço de repetição aninhado com complexidade $O(n^2)$ que “traduz” todo o conteúdo que foi salvo na matriz em novos caracteres e os desenha na tela, incluindo as entidades que estão posicionadas, tornando o mapa legível e compreensível para o usuário e criando a lógica visual de algo realmente jogável.

3.3 Jogador

A existência do jogador se inicia na criação do *struct Player*, que informa a sua posição vertical e horizontal na matriz.

O jogador é representado pelo caractere “@” no mapa, onde precisa digitar W, S, A ou D no console para se movimentar para cima, baixo, esquerda ou direita, respectivamente. O personagem tem interações com todos os elementos presentes no mapa, sejam eles:

- a) a moeda, que aumenta a pontuação em um ponto na *struct Player* quando é coletada;
- b) o policial, que encerra o jogo chamando a função *gameOver()* caso ambos estejam na mesma posição da matriz (simulando a colisão);
- c) o carro, que é uma terceira entidade e só aparece no mapa após o jogador alcançar 10 pontos;
- d) a parede, que impede que o jogador avance e faz com que o policial se movimente na direção dele caso seja feita uma tentativa de avançar contra ela, em todos os turnos com movimentação é verificado se a direção que o jogador deseja ir é válida, e isso é feito a partir do auxílio das funções *MoverParaCima()*, *MoverParaBaixo()*, *MoverParaEsquerda()*, *MoverParaDireita()*, que são funções que

possuem em seu conteúdo uma estrutura *if/else* que verifica se aquela posição na matriz possui uma parede ou policial, mantendo a estrutura do mesmo cálculo mencionado no tópico 3.2 deste relatório, porém removendo ou adicionando 1 nas posições X ou Y como comparativo nas condições do bloco.

Além das funções de movimentação, também é executada no mesmo turno a função `verificarlocal()`, que verifica se existe uma moeda na posição atual do jogador. Caso exista, a pontuação é aumentada em 1 e ela deixa de existir no mapa logo em seguida, sendo substituída por um espaço em branco transitável, que é representado por um ponto.

3.4 Policial

A entidade policial é iniciada a partir da criação da sua *struct*, onde é registrada as posições X e Y, que representam as posições vertical e horizontal, além da última movimentação realizada por ele (representada por números de 1 a 4, sendo 1 cima, 2 baixo, 3 esquerda e 4 direita).

Assim como o jogador, o policial tem uma função dedicada a dar lógica à sua movimentação, que é a *moverPolicial()*. Ela é uma função extensa, com 19 linhas, que descreve todas as possibilidades de movimentação do personagem em vários *if/else*, comparando a posição atual do jogador com a posição do policial e fazendo com que um bloco de código específico da função seja executado a depender da diferença das posições X e Y entre o jogador e ele.

3.5 Carro

A entidade carro é representada no código com a *struct* *Saida*, que declara as posições X e Y em que vai aparecer após o requisito mínimo (jogador com 10 pontos) ser atendido. Os números das variáveis estão negativos para que não apareçam visualmente para o usuário até a condição mínima ser atendida.

Após o jogador alcançar 10 pontos, condição que é verificada com um bloco *if* no final da função *while* do *main.c*, são declaradas as posições do carro, sendo elas X13 e Y17. Abaixo desta função existe outro bloco *if*, que é executado caso o jogador

esteja na exata posição do carro. Este bloco executa a função fuga(), que mostra uma mensagem de vitória ao jogador e solicita para que o mesmo digite 0 para encerrar o programa, esta mensagem é repetida em looping até que o usuário digite 0. Após, a memória alocada é liberada e o programa é encerrado corretamente.

4 MOVIMENTAÇÃO

A lógica de movimentação aplicada tem forma diminuindo ou aumentando os valores X ou Y de cada entidade a depender da direção desejada, aplicando condições ao bloco if que impedem um movimento inválido (contra paredes ou para fora do mapa).

4.1 Jogador

Como já mencionado, as funções de movimentação dos personagens possuem várias condições para que um turno realmente seja válido. Para termos noção de como funciona a movimentação do jogador, destrincharemos a função `MoverParaCima()`; `"if((c->posicao_x - 1) >= 0 && m[(c->posicao_x - 1) * largura + c->posicao_y] != PAREDE && m[(c->posicao_x - 1) * largura + c->posicao_y] != POLICIAL){ c->posicao_x++; verificarLocal(c, m);}"`.

O segmento `"if((c->posicao_x - 1) >= 0"` verifica a posição vertical do jogador. Caso seja menor que 0, a variável `posicao_x` não é incrementada e o jogador não se move. Isso evita vazamento de memória, já que impede que o jogador acesse memória indevida.

Já `m[(c->posicao_x - 1) * largura + c->posicao_y] != PAREDE` verifica se a futura posição vertical do jogador ocupa o mesmo local de uma parede, a condição seguinte mas o mesmo, mas comparando com a posição do policial. Após, é executada a função `verificarLocal()`, já mencionada anteriormente neste artigo.

4.2 Policial

A movimentação do policial abrange em pequena escala como funciona um mecanismo simples de *pathfinding*, utilizando um conceito chamado *Distância de Manhattan*.

O conceito de *Distância de Manhattan* (também conhecido como métrica do táxi) calcula a distância entre dois pontos a partir das suas coordenadas, e isso é feito comparando as coordenadas do policial e do jogador em uma grande função chamada `moverPolicial()`, na qual será destrinchada a seguir.

Dissecando o trecho “*if(pm->posicaoX > c->posicaoX && m[(pm->posicaoX - 1) * largura + pm->posicaoY] != PAREDE && pm->ultimoMovimento != 2){pm->posicaoX-;pm->ultimoMovimento = 1}*” temos a comparação da coordenada X do policial com o do jogador, após, temos a futura posição vertical do policial (cima) sendo utilizada como parâmetro com a matriz sendo escrita de forma extensa e localizando ele no mapa e verificando se existe uma parede na posição mencionada. Após essas duas verificações, é checado se o último movimento do policial foi para baixo. A verificação do último movimento evita que o policial se mova em loop tentando acompanhar o jogador, fazendo com que ele não repita movimentos e torne a perseguição maçante e sem sentido.

Caso todas as condições sejam atendidas, a coordenada X do policial é diminuída, fazendo com que ele vá para cima e registra na variável *ultimoMovimento* que ele acabou de ir para cima. Verificações com a mesma lógica são feitas no decorrer de toda a função, garantindo com que o policial persiga o jogador implacavelmente e simulando uma mini IA no programa.

5 GAME OVER

Quando a posição do policial e do jogador é a mesma (condição verificada num bloco if no final do bloco while principal do programa), é executada uma função chamada `gameOver()`.

Esta função libera a memória alocada, reabre o arquivo do mapa e mostra ao usuário, em loop, um menu de escolhas, onde ele digita 1 para tentar novamente ou 0 para encerrar o programa.

Caso o número 1 seja digitado, o jogador e o policial voltam para suas posições originais. Após, todo o mapa é lido novamente a partir do arquivo de texto e é alocada memória o suficiente no programa para que o usuário tente jogar novamente. Caso seja digitado 0, é mostrada uma mensagem e o programa é encerrado.

6 JUSTIFICATIVA DAS ESCOLHAS

A escolha do tema veio de uma certa incerteza acerca da estrutura que seria requisitada na avaliação, que ainda não tinha sido 100% anunciada e se consolidou como uma grande simulação tendo ovelhas, lobos e caçadores que interagem entre si como tema central. Pensando em encaixar o projeto nos requisitos mínimos e criar algo criativo, foi decidido criar um jogo interativo em que o jogador é perseguido por algum personagem até completar um objetivo.

Dentre muitas outras opções que envolveriam conceitos mais complexos (como um sistema de névoa de guerra atribuído à entidade perseguidora) foi selecionado um jogo simples de polícia e ladrão, onde a ideia em si é simples e tem potencial para tornar algo divertido de fato.

Implantar uma inteligência artificial no policial exige que o jogador encontre meios de utilizar os obstáculos do mapa ao seu favor para despistá-lo. Sendo um projeto pequeno, é esperado que ele não seja muito monótono e que tenha um certo nível de dificuldade, e a implementação de um jogo interativo atenderia a esses requisitos facilmente. O método de perseguição do policial foi escolhido por sua simplicidade e eficácia, sendo equivalente ao nível de conhecimento do grupo e mantendo o nível de qualidade exigido.

7 PRINCIPAIS DIFICULDADES

Uma das principais dificuldades se deu pela falta de prática envolvendo os conceitos utilizados nesse projeto, tratando-se principalmente da leitura de arquivos, manipulação de memória e a lógica de perseguição de policial. As combinações das estruturas utilizadas tornaram o projeto mais extenso e complexo que o esperado.

Ademais, o que tornou tudo ainda mais complicado de assimilar foi o aprendizado do conceito de matriz unidimensional simulando a estrutura de uma matriz bidimensional. A manipulação dessas informações teve seu ápice na construção da lógica de perseguição do policial, onde ele andava para os lados repetidamente tentando alcançar o jogador, ficando preso entre as paredes. O ajuste desse comportamento levou a criação de uma função de 32 linhas, dedicadas somente a movimentação do policial.

REFERÊNCIAS

Depth First. Manhattan Distance | Visually Explained. Youtube, 2025. Disponível em <https://youtu.be/s7ssfa5DhtI?is=Ymqe68rmpbluoPCz>. Acesso em: 18 mai. 2026

MOREIRA, Jander. Alocação dinâmica de memória. 2013.